

CS 451 – Homework 2

Reinforcement Learning: Mars Rover

Umut Ardil Murat
Student ID: S028641

Spring 2026

1 Introduction

In this homework, I implemented two reinforcement learning approaches for a Mars Rover grid-world environment. The rover moves on a grid, tries to reach sample collection sites with positive rewards, and should avoid craters with negative rewards. The environment is stochastic because the rover may slip sideways instead of moving exactly in the intended direction.

The first part of the homework uses Value Iteration. This is a model-based method, so it uses the full MDP information such as states, legal actions, transition probabilities, and rewards. In the later parts, I implemented Q-Learning, which is model-free. In Q-Learning, the rover does not know the transition function beforehand and instead learns from the transitions it experiences during training.

The main difference I observed while implementing them is that Value Iteration updates values by looking at all possible next states from the transition model, while Q-Learning updates only the state-action pair that was actually experienced.

2 Q1: Value Iteration

Value Iteration is an offline planning algorithm. Since the transition model is available, the agent can repeatedly update the estimated value of each state before actually acting in the environment.

2.1 Algorithm

For each state, the algorithm applies the Bellman update:

$$V_{k+1}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

Here, $T(s, a, s')$ is the probability of reaching state s' after taking action a in state s , $R(s, a, s')$ is the reward for that transition, and γ is the discount factor. After several iterations, the values become closer to the optimal values, and the best action for a state can be selected by comparing the Q-values of the legal actions.

2.2 Implementation Details

I implemented the main Value Iteration logic in `value_iteration_agents.py`. The important methods were:

- `run_value_iteration()`: This method performs the main iteration loop. For each sweep, I create a new `Counter` object and store the updated values there. After all states are processed, I replace `self.values` with the new values. This is important because Value Iteration should use the values from the previous iteration while computing the next iteration.
- `compute_q_value_from_values(state, action)`: This method computes the Q-value for a given state and action. I used the MDP transition function to get all possible next states and their probabilities. Then I accumulated the expected value using the immediate reward and the discounted value of the next state.
- `compute_action_from_values(state)`: This method finds the best action for a state by computing the Q-value of every legal action and returning the action with the highest value. If the state is terminal and has no legal actions, the method returns `None`.

One mistake I made at first was updating `self.values` directly inside the loop. This caused later states in the same sweep to use already updated values, which is not the intended batch update behavior. After switching to a separate `Counter` for each iteration, the value updates became consistent.

3 Q2: Mission Profiles (Canyon Grid)

In this question, I selected different parameter settings for the Canyon Grid. This grid contains a close exit with reward +1, a distant depot with reward +10, and craters with reward -10. The goal was to produce different policies by changing the discount factor, noise, and living reward.

3.1 Parameter Analysis

The three parameters affect the rover's behavior in different ways:

- **Discount factor (γ)**: This controls how much the agent cares about future rewards. A low discount makes the agent prefer rewards that are closer, while a high discount makes the distant +10 reward more attractive.
- **Noise**: This controls how risky movement is. When noise is 0, the rover always moves in the intended direction. In that case, paths close to the crater are safe. When noise is higher, moving next to the crater becomes risky because the rover can slip sideways.
- **Living reward**: This is the reward received at each non-terminal step. A negative living reward encourages the rover to finish quickly. A positive living reward can make the rover prefer staying alive instead of reaching a terminal state.

I checked the resulting policies by looking at whether the arrows moved toward the close exit or the distant depot, and whether the selected path stayed near the crater row or avoided it.

3.2 Chosen Parameters

Table 1 shows the parameter values I used for the five required behaviors.

For part (e), the important point is that the discounted return from receiving a living reward of +1 forever is:

$$1 + 0.9 + 0.9^2 + \dots = \frac{1}{1 - 0.9} = 10$$

	Behavior	γ	Noise	Living Reward	Reasoning
(a)	Close exit, risk cliff	0.2	0.0	0.0	The low discount makes the nearby +1 exit more attractive than the distant +10 reward. Since noise is 0, the path near the crater is not risky.
(b)	Close exit, avoid cliff	0.2	0.2	0.0	The low discount still makes the close exit preferable, but the non-zero noise makes the crater path dangerous, so the policy avoids it.
(c)	Distant depot, risk cliff	0.9	0.0	0.0	The high discount makes the distant +10 depot valuable enough to pursue. Since there is no noise, the rover can safely take the shorter risky-looking path.
(d)	Distant depot, avoid cliff	0.9	0.2	0.0	The high discount still favors the distant depot, but the noise makes the path near the crater risky, so the rover chooses a safer route.
(e)	Never terminate	0.9	0.2	1.0	The positive living reward makes continuing to move attractive, so the rover avoids both exits and craters.

Table 1: Parameter settings for the five Canyon Grid policies.

Because this continuing reward is very attractive compared to terminating, the agent can prefer avoiding terminal states.

4 Q3: Q-Learning

Q-Learning is different from Value Iteration because it does not use the transition model. Instead of computing expectations over all possible next states, the agent updates its Q-values from the transitions it actually observes.

4.1 Algorithm

For a transition (s, a, r, s') , the Q-Learning update is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

In this formula, α is the learning rate. The term

$$r + \gamma \max_{a'} Q(s', a')$$

is the sample target. The current Q-value is moved toward this target. If the new experience is better than expected, the Q-value increases. If it is worse than expected, the Q-value decreases.

4.2 Implementation Details

For the core Q-Learning part, I implemented four main methods in `qlearning_agents.py`:

- `get_q_value(state, action)`: This returns the Q-value for a state-action pair. Since I used a `Counter`, unseen pairs automatically have value 0.
- `compute_value_from_q_values(state)`: This computes $V(s)$ by taking the maximum Q-value over all legal actions. If the state has no legal actions, it returns 0.
- `compute_action_from_q_values(state)`: This returns the action with the highest Q-value. I also handled ties by choosing randomly among the best actions. This matters at the beginning of training because many Q-values are equal to 0.
- `update(state, action, next_state, reward)`: This applies the Q-Learning update rule. I first compute the old Q-value, then compute the sample target using the reward and the value of the next state, and finally update the Q-table.

A small but important detail is that unseen actions have Q-value 0. This means an untried action can still be selected if the tried actions have negative Q-values. This behavior is useful because it gives the agent a chance to discover better actions.

5 Q4: Epsilon-Greedy Exploration

If the agent always chooses the action with the highest current Q-value, it may stop exploring too early. This can be a problem because early Q-values are not reliable. To solve this, I implemented epsilon-greedy action selection in the `get_action` method.

The rule is:

- With probability ϵ , choose a random legal action.
- With probability $1 - \epsilon$, choose the best known action according to the current Q-values.

In my implementation, I used `util.flip_coin(self.epsilon)` to decide whether the agent should explore. If it explores, I use `random.choice` on the legal actions. Otherwise, I call the method that returns the best action from the current Q-values.

This same implementation also works for the Crawler robot domain. I did not need to change the Q-Learning logic because the algorithm treats states and actions as keys in a table. In the grid world, states are coordinate-like values, while in the Crawler domain they represent robot configurations. Since the Q-table only needs hashable state-action pairs, the same code can be reused.

6 Q5: Q-Learning Deployment

In the final part, the Q-Learning agent is trained on `small_grid`. The given hyperparameters are:

$$\epsilon = 0.05, \quad \alpha = 0.2, \quad \gamma = 0.8$$

The agent is trained for 2000 episodes. After training, testing is done with $\epsilon = 0$, so the agent stops exploring and only follows the learned policy.

In my test runs, the agent reached a win rate above the required threshold on the small grid after training. This shows that the Q-values learned during the 2000 episodes were enough to produce a reasonable policy for this small environment.

However, this result does not mean that tabular Q-Learning scales well to all environments. In tabular Q-Learning, every state-action pair has its own separate value. The agent does not generalize from one state to another. Because of this, larger grids or more complex environments would require many more episodes, since the agent would need to visit many different state-action pairs enough times.

The Crawler test also shows the benefit of keeping the implementation general. The same update rule and action selection code can be used in a different domain as long as the environment provides legal actions, rewards, and next states.

7 Conclusion

In this homework, I implemented both Value Iteration and Q-Learning. Value Iteration used the full MDP model and updated all states using Bellman backups. The most important implementation detail there was using batch updates instead of changing the values in place during a sweep.

Q-Learning learned from experience without using the transition model. The main parts were storing Q-values, computing the best action from those values, applying the temporal difference update, and using epsilon-greedy exploration. I also saw why exploration matters, especially at the start of training when many Q-values are still equal.

Overall, this homework made the difference between model-based planning and model-free learning clearer. Value Iteration can plan directly from the model, while Q-Learning needs interaction with the environment to gradually improve its policy.